University of Bologna - DIFA

Triton ^{3}H Ground State computation

using Similarity Renormalization Group in the No-Core Shell Model

Instructor:
Prof. Paolo Finelli

Author:
Andrea Nicoletti

April 2026

Table of Contents

1. No-Core Shell Model
2. Similarity Renormalization Group
3. Code

No-Core Shell Model

- The NCSM is an ab initio approach treating all nucleons as active.
- It treats all nucleons as active unlike traditional shell.
- It uses a many-body intrinsic hamiltonian

$$H = \sum_{i=1}^{A} \frac{p_i^2}{2m} - \frac{P_{\text{cm}}^2}{2Am} + \sum_{i<j} V_{ij} + \sum_{i<j<k} V_{ijk} \tag{1}$$

  in a H.O. basis truncated by a maximum number of excitations

$$\sum_{i=1}^{A} (2n_i + l_i) - N_0 \leq N_{\text{max}}$$

  results improve as N_{max} increases.
- Applicable primarily to light nuclei due to computational complexity.

No-Core Shell Model

- Working a truncated model space P (with complement Q) it must construct a effective hamiltonian acting only in P

$$H_{\text{eff}} = Pe^{-S}He^{S}P$$

- The Lee-Suzuki construction uses an operator satisfying

$$Qe^{-S}He^{S}P = 0$$

- We approximate a two-body effective interaction

$$V_{\text{eff}}^{(2)} = P_2 e^{-S_2}(T + V)e^{S_2}P_2 - P_2 T P_2$$

using two nucleons at a time.

Similarity Renormalization Group

- The SRG is a framework in quantum many-body physics used to decouple low and high momentum scales by continuously trasforming the system's hamiltonian to a simpler form.
- It introduces a flow equation

$$\frac{dH(s)}{ds} = [\eta(s), H(s)]$$

  where $H(s) = U(s)H(0)U^{+}(s)$ is the parametrized Hamiltonian and $\eta(s)$ is the anti-hermitian generator.
- As the parameter s increases off-diagonal components are suppressed and the hamiltonian reaches a band-diagonal form.
- In the process physical observables remain invariant due to unitarity.

Similarity Renormalization Group

- There are different choices for the flow's generator

$$\eta(s) = \frac{dU(s)}{ds} U^{+}(s) = -\eta^{+}(s)$$

- The hamiltonian is split as

$$H(s) = H_d(s) + H_{od}(s), \quad H(s) \xrightarrow[s\to\infty]{} H_d(s),\ H_{od}(s) \xrightarrow[s\to\infty]{} 0$$

- Then the Wegner's choice of $\eta(s)$

$$\eta(s) = [H_d(s), H(s)]$$

suppresses off-diagonal elements as $(H_{od})_{ij} \sim e^{-(E_i - E_j)^2 s}$.

- Formally SRG is basis-independent but we need to make a choice when splitting the hamiltonian and to represent it in matrix form.

SRG and ^3H

- The initial hamiltonian of the triton ^3H is composed by a kinetic energy term T and a chiral 2+3 nucleon interaction V with matrix elements given in the 3-nucleon Jacobi-cordinates H.O. basis.
- We choose

$$\eta(s) = [T, H(s)]$$

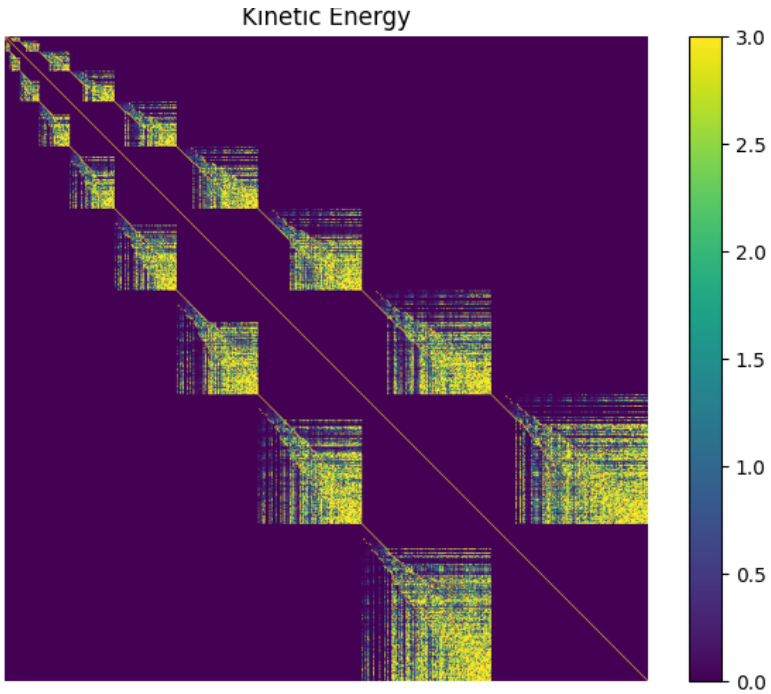
- The flow equation becomes

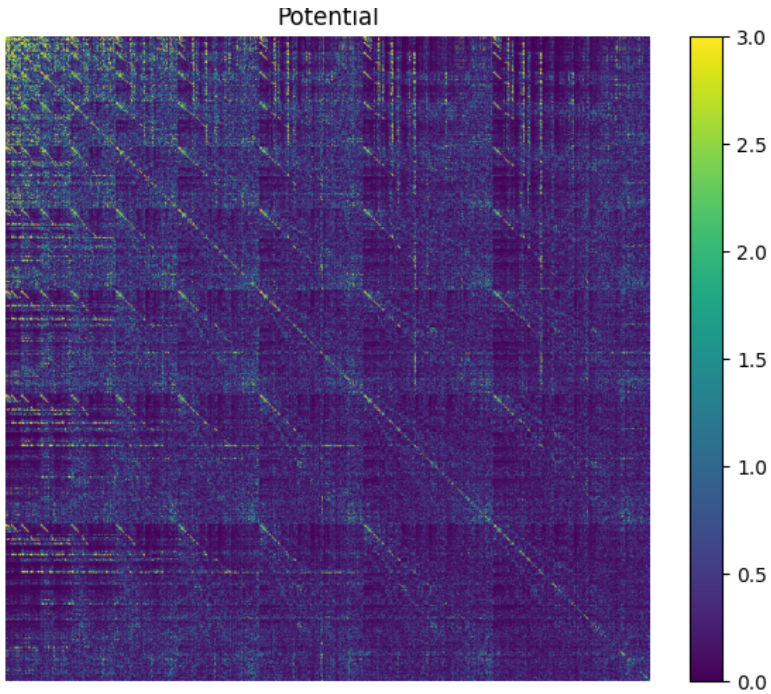
$$\frac{dH(s)}{ds} = M_n^2 \left[TH(s)^2 - 2H(s)TH(s) + H(s)^2 T \right] \tag{2}$$

where $M_n = (m_p + m_n)/2\hbar^2$ adimensionalizes the equation.

SRG and ^3H

- The initial matrix elements of T and V are:





Flow Evolution

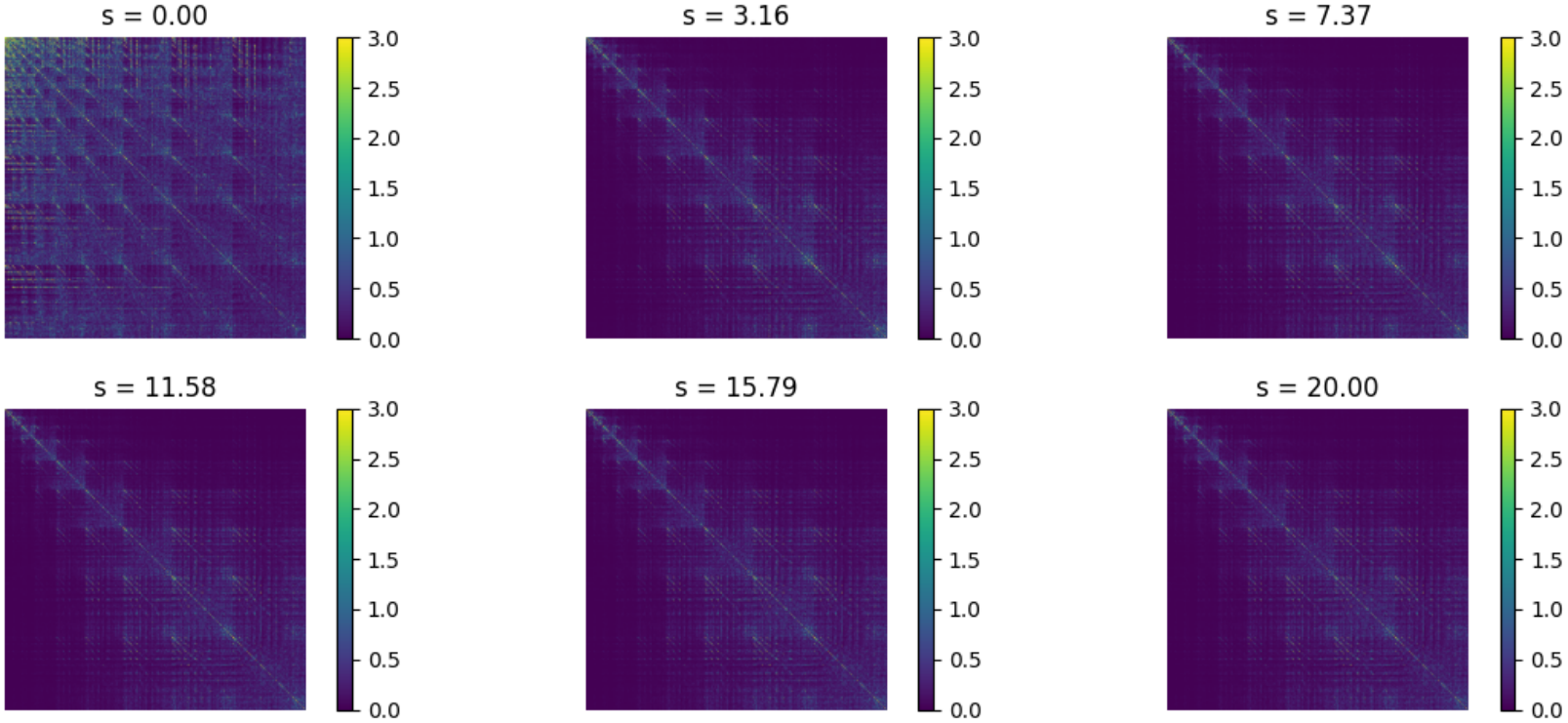
- The flow equation (2) is a nonlinear differential equation which cannot be solved analitically for $H(s)$ therefore we have to integrate it numerically.
- We choose to use the Euler's algorithm, a first order integrator.
- Its update eq. for $\dot{x} = A(x)$ is $x_{n+1} = x_n + h\,A(x_n)$, where h is the chosen timestep.
- Therefore the discretized flow dynamics is

$$H_{n+1} = H_n + h\,M_n^2\left[TH_n^2 - 2H_n TH_n + H_n^2 T\right]$$

  where $H_n = H(nh)$ and $s = hn$.
- We set $h = 0.0001$ and run 200000 steps.

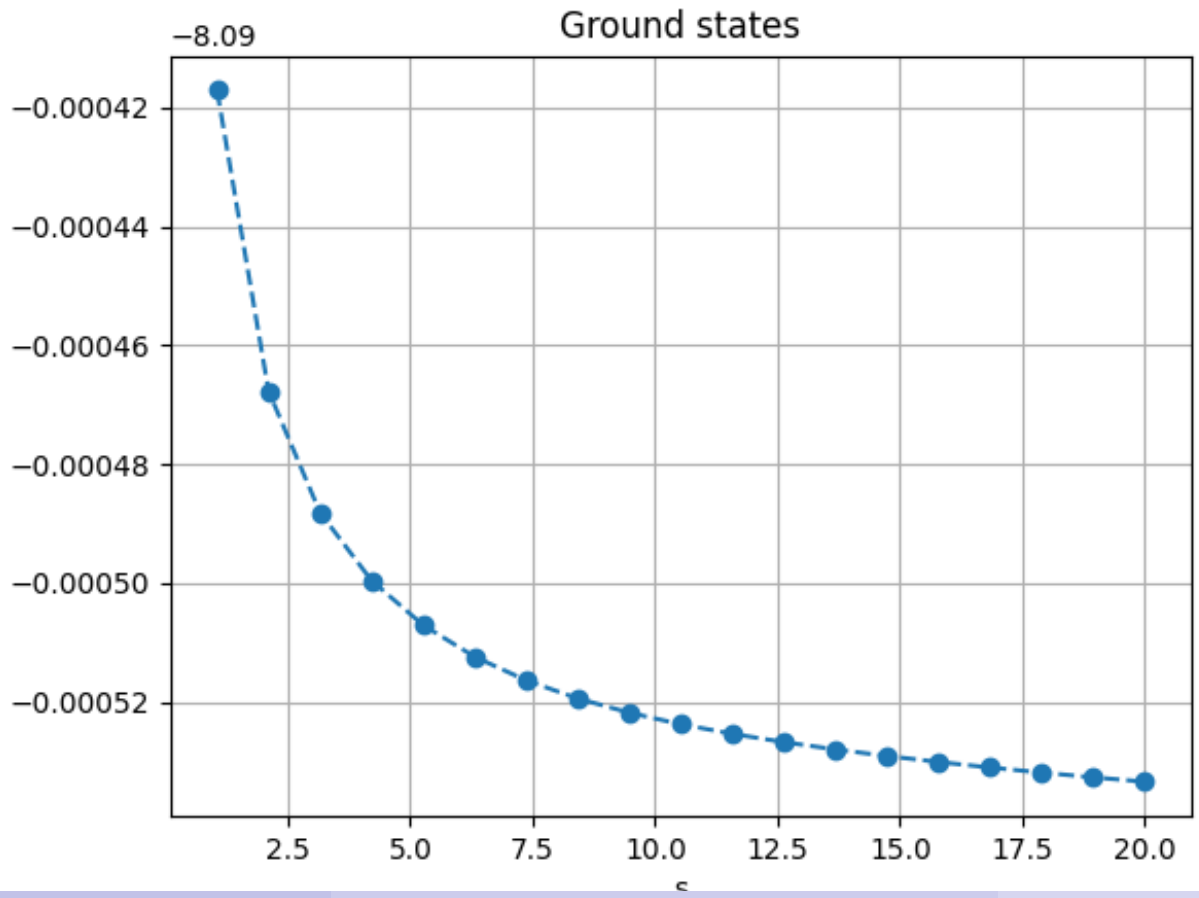
Flow Evolution

- Some instances of the interaction as the evolution progresses:



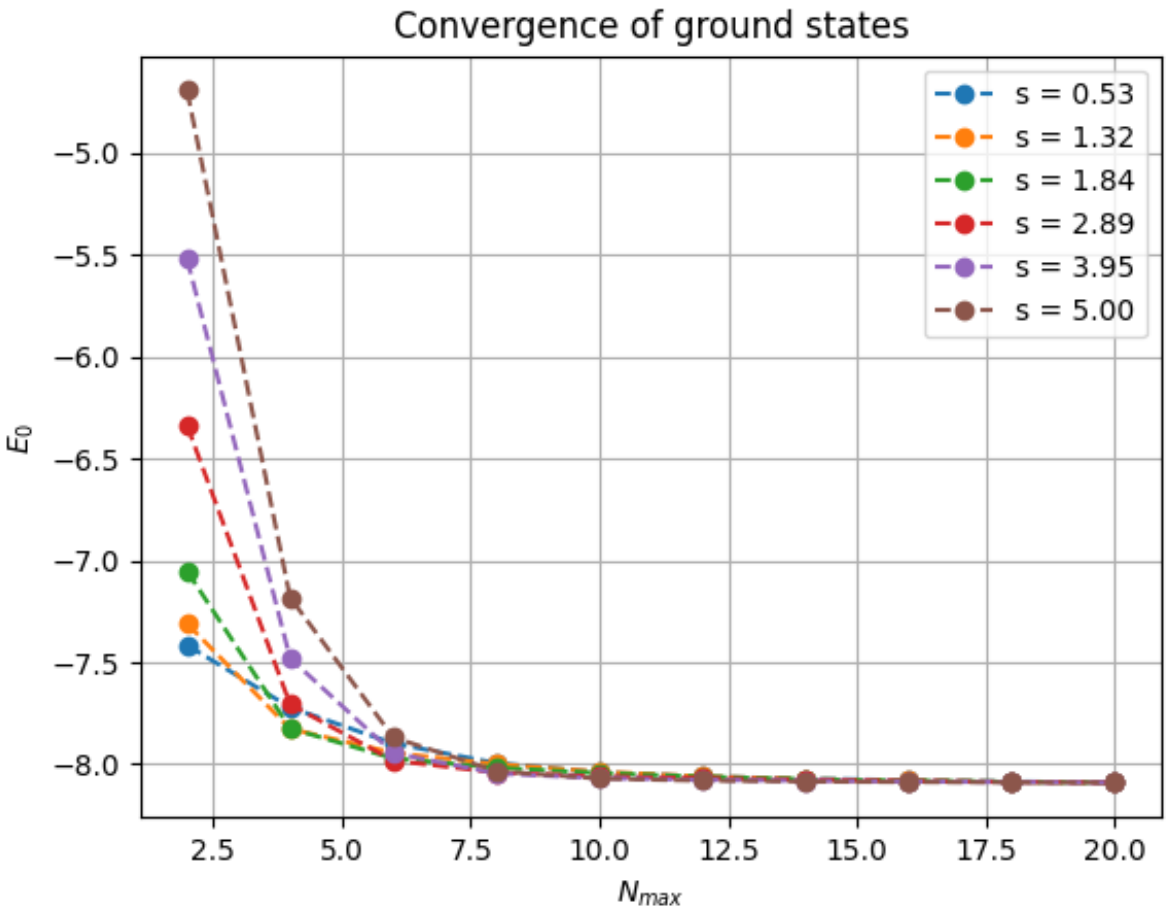
Ground energy

- After applying SRG to decouple low and high energy states and soften interactions, it becomes just a matrix eigenvalue problem.
- The resulting ground state computed at different flow parameters values are:



Convergence

- As mentioned before, results should improve as $N_{max} \to \infty$, we can therefore check convergence for values in $[0, 20]$:



Euler Integrator *(https://github.com/andreanicoletti/triton-ground-state)*

```python
class EulerIntegrator:

    def __init__(self, A, dt):
        self.A = A
        self.dt = dt

    def step(self, x, a, t) -> tuple[NDArray, NDArray, float]:
        t_new = t + self.dt
        x_new = x + self.dt * a
        a_new = self.A(x_new, t_new)
        return x_new, a_new, t_new # we also return the acceleration for more efficient
             computations

    def evolve(self, x0, n_steps, checkpoints=None, verbose=False, init_msg=None, end_msg
         =None) -> tuple[NDArray, NDArray]:
        checkpoints = checkpoints if checkpoints is not None else n_steps

        # save_interval = max(1, n_steps // (checkpoints - 1)) # one checkpoint is
             already at the initial time

        save_indices = np.linspace(1, n_steps, checkpoints, dtype=int)
        if verbose:
            print("checkpoints:")
            print(save_indices)

        pos = np.zeros((checkpoints+1, *x0.shape))
        times = np.zeros(checkpoints+1)
```

Euler Integrator *(https://github.com/andreanicoletti/triton-ground-state)*

```
        pos[0] = x = x0
        times[0] = t = 0.0
        a = self.A(x0, 0.0)

        if init_msg is not None:
            print(init_msg)

        k = 1
        for i in tqdm(range(1, n_steps+1), disable=not verbose):
            x, a, t = self.step(x, a, t)

            if i in save_indices:
                pos[k] = x
                times[k] = t
                k += 1

        if end_msg is not None:
            print(end_msg)

        return pos, times
```

Visualization *(https://github.com/andreanicoletti/triton-ground-state)*

```python
def plot_matrix(matrices, titles=None, shape=None, clip=None, figsize=None, cmap=None):
    if shape is None:
        # automatically set a square grid
        ncols = int(np.ceil(np.sqrt(n)))
        nrows = int(np.ceil(n / ncols))
    else:
        nrows, ncols = shape

    fig, axes = plt.subplots(nrows, ncols, figsize=figsize)
    axes = np.array(axes).reshape(-1)

    vmin, vmax = clip if clip is not None else (None, None)
    cmap = cmap if cmap is not None else 'viridis'
    if titles is None:
        titles = [f"Matrix␣{i}" for i in range(n)]

    for i, mat in enumerate(matrices):
        im = axes[i].imshow(np.abs(mat), vmin=vmin, vmax=vmax, cmap=cmap, origin='upper')
        axes[i].set_title(titles[i])
        fig.colorbar(im, ax=axes[i])

    # we hide axis since they are not very indicative of the basis the matrix is in
    for j in range(len(axes)):
        axes[j].set_axis_off()

    fig.tight_layout()
    return fig
```

Analysis *(https://github.com/andreanicoletti/triton-ground-state)*

```
# we set some needed constants
hbarc = 197.327 # [MeV fm]
MN0 = (938.27231 + 939.56563) / 2 # average nucleon mass [MeV]
MN = MN0 / hbarc**2 # average nucleon mass in units [MeV^-1 fm^-2]

# we import the matrix elements from the data files and display them
%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt
from visualization import plot_matrix

TMat_path = './data/kinetic_EMax20_hw20.dat'
VMat_path = './data/chi2b3b_EMax20_hw20.dat'

TMat = np.loadtxt(TMat_path)
VMat = np.loadtxt(VMat_path)
print(f"Kinetic energy - shape: {TMat.shape}, max: {TMat.max()}, min: {TMat.min()}")
print(f"Potential - shape: {VMat.shape}, max: {VMat.max()}, min: {VMat.min()}")

fig = plot_matrix(
    [TMat, VMat],
    titles=["Kinetic Energy", "Potential"],
    shape=(1, 2),
    clip=(0, 3),
    figsize=(12, 5)
)
plt.show()
fig.savefig('./figs/init_matrix.png')
```

Analysis *(https://github.com/andreanicoletti/triton-ground-state)*

```python
# we run a integrator of the flow equations
from euler_integrator import EulerIntegrator
import matplotlib.pyplot as plt
from visualization import plot_matrix

# define the flow equation derivative
def flow_var(Hn, t):
    prod = (TMat @ Hn @ Hn) - (2 * Hn @ TMat @ Hn) + Hn @ Hn @ TMat
    return (MN**2) * prod

integrator = EulerIntegrator(flow_var, 0.0001)
points, times = integrator.evolve(TMat + VMat, 200000, checkpoints=20, verbose=True,
     init_msg="Evolving␣SRG␣flow")

# plot some of the flow matrices (isolating the interaction)
plt_ind = [0, 4, 8, 12, 16, 20]
plt_mat = points[plt_ind]
plt_t = times[plt_ind]

fig = plot_matrix(
    matrices=[(mat - TMat) for mat in plt_mat],
    titles=[f"s␣=␣{s:.2f}" for s in plt_t],
    shape=(2, 3),
    clip=(0, 3),
    figsize=(12, 5)
)

plt.show()
fig.savefig('./figs/flow_matrix.png')
```

Analysis *(https://github.com/andreanicoletti/triton-ground-state)*

```
# Finally we find the lowest energy eigenvalue by diagonalizing the resulting matrix
import matplotlib.pyplot as plt

ground_states = []
for mat in points[2:]:
    eigenvalues, eigenvectors = np.linalg.eigh(mat)
    ground_states.append(eigenvalues[0])

plt.plot(times[2:], ground_states, linestyle='--', marker='o')
plt.title("Ground␣states")
plt.xlabel("s")
plt.ylabel("$E_0$")
plt.grid(True)
plt.savefig('figs/ground_states.png')
plt.show()

print(f"s␣=␣{times[-1]}")
print(f"ground␣state␣=␣{ground_states[-1]}")
```

Analysis *(https://github.com/andreanicoletti/triton-ground-state)*

```
# We can check convergence for different values of N_max
n_maxs = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20];
dims = [5, 15, 34, 64, 108, 169, 249, 351, 478, 632];

plt_ind = [3, 6, 8, 12, 16, 20]
plt_mat = points[plt_ind]
plt_t = times[plt_ind]

conv_list = []
for mat in plt_mat:
    dim_list = []
    for dim in dims:
        H_sub = mat[:dim, :dim]
        eigenvalues, _ = np.linalg.eigh(H_sub)
        dim_list.append(eigenvalues[0])
    conv_list.append(dim_list)


# plot convergence of g.s. for different flow params
for i, time in zip(conv_list, plt_t):
    plt.plot(n_maxs, i, linestyle='--', marker='o', label=f"s = {time:.2f}")

plt.title("Convergence of ground states")
plt.xlabel(r"$N_{max}$")
plt.ylabel(r"$E_0$")
plt.grid(True)
plt.legend()
plt.savefig('figs/conv_lists.png')
plt.show()
```